

Multivariate analysis exam

UCT Statistics Honours 2026

Table of contents

Questions	1
Analysis I	1
Question 1.1	2
Analysis II	4
Question 2.1	4
20 principal components were selected to reconstruct the image. The cumulative variance explained by the first 20 components was approximately 92%. This yielded a blurry image where one can see what was photographed by finer details are omitted. The figure in (b) contained all the detail.	8
Analysis III	8
Question 3.1	9
Question 3.2	11
Question 3.3	15
Question 3.4	21
Multiple choice	25

Questions

Analysis I

```
if (!exists("read_exam_data", mode = "function")) {  
  stop("Please run the first chunk to define the 'read_exam_data' function.")  
}  
data_tidy_stress <- read_exam_data("data_tidy_stress.csv")
```

```

Rows: 500 Columns: 6
-- Column specification -----
Delimiter: ","
dbl (6): age, income, satisfaction, experience, hours, stress

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

Question 1.1

Part a

```

# Calculate correlation matrix:
cor_mat = cor(data_tidy_stress)
round(cor_mat,4)

```

	age	income	satisfaction	experience	hours	stress
age	1.0000	0.8327	0.6392	0.9682	0.0637	0.0226
income	0.8327	1.0000	0.7284	0.8057	0.0622	0.0406
satisfaction	0.6392	0.7284	1.0000	0.6269	-0.1606	-0.2541
experience	0.9682	0.8057	0.6269	1.0000	0.0607	0.0153
hours	0.0637	0.0622	-0.1606	0.0607	1.0000	0.7349
stress	0.0226	0.0406	-0.2541	0.0153	0.7349	1.0000

Part b

```

# Eigen decomposition:
eig_mat = eigen(cor_mat)
eig_vec = as.matrix(eig_mat$vectors)
eig_val = eig_mat$values

# Loadings = eigen vectors:
loadings = eig_vec[,1:2]
round(loadings,4)

```

	[,1]	[,2]
[1,]	-0.5225	0.0727
[2,]	-0.5084	0.0626
[3,]	-0.4478	-0.1875
[4,]	-0.5167	0.0690

```
[5,] -0.0005  0.6862
[6,]  0.0316  0.6928
```

Part c

```
# Obtain loadings using ML approach:
fa_fit = factanal(data_tidy_stress,factors=2)
print(round(fa_fit$loadings,4),cutoff=0)
```

Loadings:

	Factor1	Factor2
age	0.992	0.058
income	0.836	0.070
satisfaction	0.663	-0.232
experience	0.972	0.050
hours	0.022	0.738
stress	-0.035	0.997

	Factor1	Factor2
SS loadings	3.069	1.603
Proportion Var	0.512	0.267
Cumulative Var	0.512	0.779

Part d

Variables age,income,satisfaction and experience load strongly onto factor 1. Variables hours and stress load strongly onto factor 2. Factor 1 represents the length of time a person has been working for as well as their expertise. Factor 2 represents workload and how much energy is being expended into the job.

Part e

PCA aims to maximise variation. This is done by exploiting the correlation of variables with each other. Strongly correlated variables will load strongly onto the same components. The additional of an additional variable uncorrelated with the existing variables will not greatly change the first two components but rather load strongly onto the third.

Part f

Incorrectly calculated. Factor scores should have a variance of 1. The variance in the image is too large.

Analysis II

```
if (!exists("read_exam_data", mode = "function")) {  
  stop("Please run the first chunk to define the 'read_exam_data' function.")  
}  
img_list <- read_exam_data("pca_image_results.rds")
```

Question 2.1

Part a

Yes, this was appropriate. Pixels are measured on the same scale and thus do not need to be scaled.

Part b

```
# Reconstruction of original image:  
scores = img_list$Y  
loadings = img_list$V  
eig_val = as.matrix(img_list$lambda)  
col_means = matrix(img_list$M, nrow=2160, ncol=1215)  
  
og_img = t(t(scores)%*%t(loadings))+col_means  
  
save_img(og_img, 'og_img.png')
```

```
[1] "og_img.png"
```

```
knitr::include_graphics("og_img.png")
```



Part c

```
# Reconstruct the image using fewer PCs:
```

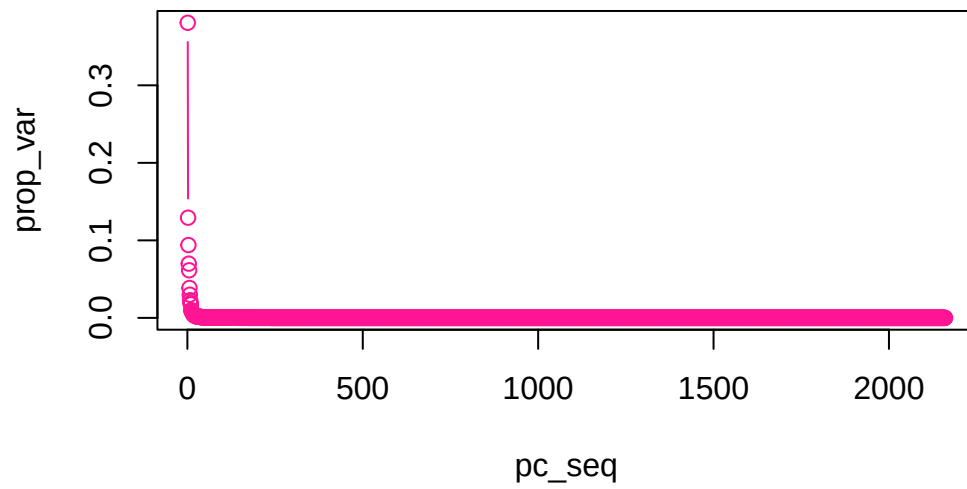
```
tot_var = sum(eig_val)
prop_var = eig_val/tot_var
cum_var = cumsum(prop_var)
pc_seq = seq(1:length(eig_val))
```

```
# Cumulative variance:
round(cum_var[1:20],4)
```

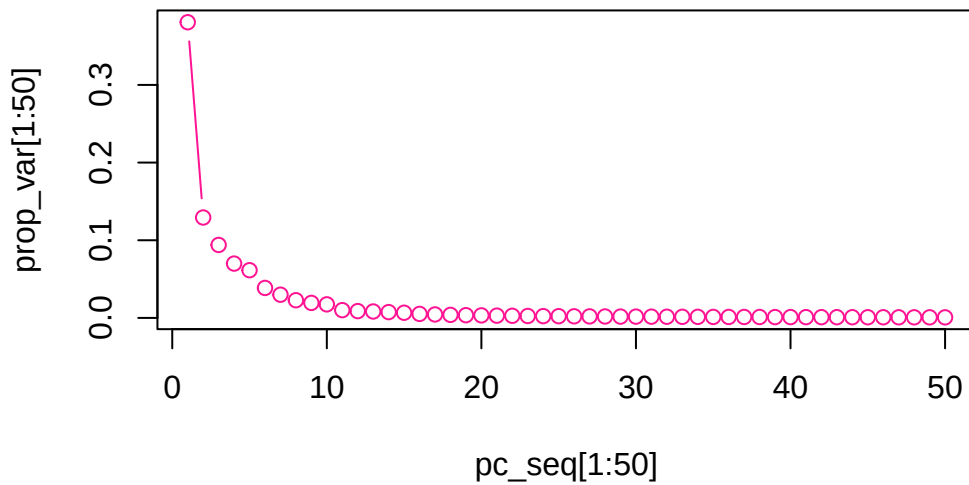
```
[1] 0.3807 0.5100 0.6039 0.6739 0.7353 0.7739 0.8038 0.8265 0.8459 0.8633
[11] 0.8734 0.8821 0.8904 0.8979 0.9046 0.9097 0.9142 0.9181 0.9216 0.9249
```

```
# Plot:
plot(pc_seq,prop_var,
     type='b',
```

```
col='deeppink')
```



```
plot(pc_seq[1:50],prop_var[1:50],  
     type='b',  
     col='deeppink')
```



As seen in the above plot there are over 2000 possible principal components.

```
# No. principal components used:
k = 20

scores_recon    = img_list$Y[1:1215,1:k]
loadings_recon  = img_list$V[,1:k] # step 1
eig_val_recon   = as.matrix(img_list$lambda)
col_means_recon = matrix(img_list$M,nrow=2160,ncol=1215)

recon_img = t(t(scores_recon)%*%t(loadings_recon))+col_means_recon)
```

The original data has 1215 “observations” with 2160 “variables”

$((1215 \times 1000) \times (1000 \times 2160)) = 1215 \times 2160$

1: cor_mat 2: eigen decomp 3: loadings = eig_vec -> this is where you select PCs 4: scores[1215:1000] = data[1215:2160]loadings[2160:1000] 5: data = t(t(scorest(loadigs))+col_means)

```
save_img(recon_img,'recon_img.png')
```

```
[1] "recon_img.png"
```

```
knitr::include_graphics("recon_img.png")
```



20 principal components were selected to reconstruct the image. The cumulative variance explained by the first 20 components was approximately 92%. This yielded a blurry image where one can see what was photographed by finer details are omitted. The figure in (b) contained all the detail.

Analysis III

```
if (!exists("read_exam_data", mode = "function")) {  
  stop("Please run the first chunk to define the 'read_exam_data' function.")  
}  
data_tidy_lum_response <- read_exam_data("data_tidy_lum_response.csv")
```

Rows: 100 Columns: 16

-- Column specification -----
Delimiter: ","


```
chr (1): pid
dbl (15): crp, ddimer, fractalkine, ifng, il10, il1a, il1ra, mcp1, mip1a, mi...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data_tidy_lum_meta <- read_exam_data("data_tidy_lum_meta.csv")
```

```
Rows: 100 Columns: 6
-- Column specification -----
Delimiter: ","
chr (6): pid, infxn, age, sex, ethnicity, household_size

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data_tidy_lum <- data_tidy_lum_meta |>
  dplyr::left_join(data_tidy_lum_response, by = "pid") |>
  dplyr::mutate(
    age_infxn = paste0(age, "_", infxn)
  ) |>
  dplyr::select(pid:household_size, age_infxn, everything())
```

Question 3.1

Part a

```
library(ca)
# Create a row-principal CA plot:
ca_tbl = table(data_tidy_lum_meta$age, data_tidy_lum_meta$ethnicity)
ca_tbl
```

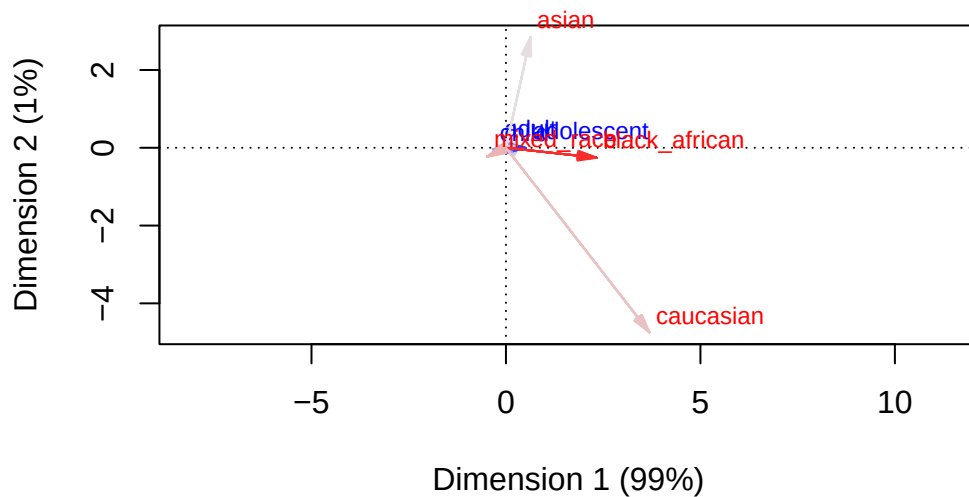
	asian	black_african	caucasian	mixed_race
adolescent	4	9	1	20
adult	3	2	0	28
child	2	1	0	30

```

P = ca_tbl/sum(ca_tbl)
ca_fit = ca(P)

plot.ca(ca_fit,
        map='rowprincipal',
        contrib='absolute',
        mass=c(TRUE,TRUE),
        arrows=c(TRUE,TRUE))

```



Part b

In the above row-principal plot, 'caucasian' lies far away from the origin suggesting that this ethnicity is highly differentiated. The same is seen with 'asian'. 'Asian' and 'black african' are approximately orthogonal suggesting starkly different profile patterns. 'mixed race' lies approximately on the origin representing the average profile.

Part c

The chi-squared distance is a weighted Euclidean distance. It is used to down-weight more abundant columns, as opposed to normal Euclidean distance which does not account for the number of observations in each column. A large distance in columns with a small number of observations is more informative than the same distance in a column with more observations.

Part d

PCA is typically used as a dimension reduction technique on continuous variables by creating linear combinations of explanatory variables. Since the above data is count data, performing PCA would create components where highly correlated variables load strongly and make it difficult to interpret the fitted model.

Question 3.2

Part a

```
# Construct response matrix:
Y = cbind(data_tidy_lum_response$crp,
          data_tidy_lum_response$ddimer)

# Construct predictor matrix:
X = cbind(1,
          data_tidy_lum_response$ifng,
          data_tidy_lum_response$il10)

# Calculate regression coefficients:
beta_hat = solve(t(X)%*%X)%*%t(X)%*%Y
rownames(beta_hat) = c('intercept', 'ifng', 'il10')
colnames(beta_hat) = c('crp', 'ddimer')
round(beta_hat, 4)
```

	crp	ddimer
intercept	1.1724	0.9362
ifng	-0.2500	0.1399
il10	-0.0474	-0.0350

Part b

```
# Fit restricted model:
Yres = cbind(data_tidy_lum_response$crp,
             data_tidy_lum_response$ddimer)

Xres = cbind(1,
             data_tidy_lum_response$il10)

beta_hatres = solve(t(Xres)%*%Xres)%*%t(Xres)%*%Yres
```

```

rownames(beta_hatres) = c('intercept','il10')
colnames(beta_hatres) = c('crp','ddimer')
round(beta_hatres,4)

```

```

          crp  ddimer
intercept 0.9302 1.0717
il10      -0.0260 -0.0469

```

```

# Calculate yhat:
yhat = X%%beta_hat
yhat_res = Xres%%beta_hatres

n = nrow(Y)
k = ncol(X)
r = ncol(Y)
p = k-1
q = ncol(X)-ncol(Xres)

# Calculate MLE:
MLE = t(Y-yhat)%%(Y-yhat)/n
MLEres = t(Yres-yhat_res)%%(Yres-yhat_res)/n

# Calculate lambda:
lambda = det(MLE)/det(MLEres)
round(lambda,4)

```

```
[1] 0.9552
```

```

# Calculate test statistic:
stat = -(n-k-0.5*(r-p+q+1))*lambda
round(stat,4)

```

```
[1] -91.6971
```

```

# Calculate p-value:
pval = pchisq(stat,r*q,lower.tail=TRUE)
round(pval,4)

```

```
[1] 0
```

The above hypothesis test suggest that ifng does affect crp or ddimer. The very small p-value provides evidence that ifng is significant in the regression model.

Part c

```
pcr_dat = data_tidy_lum_response[,-(1:2)]  
# Correlation matrix:  
cor_mat = cor(pcr_dat)  
round(cor_mat,4)
```

	ddimer	fractalkine	ifng	il10	il1a	il1ra	mcp1	mip1a
ddimer	1.0000	0.0693	0.0750	-0.0408	0.0289	-0.0161	0.0306	0.1596
fractalkine	0.0693	1.0000	-0.1503	-0.0410	-0.0407	0.1606	0.0382	-0.0116
ifng	0.0750	-0.1503	1.0000	-0.1477	-0.1316	0.0357	0.0540	-0.0735
il10	-0.0408	-0.0410	-0.1477	1.0000	-0.1219	-0.0339	0.0717	0.1435
il1a	0.0289	-0.0407	-0.1316	-0.1219	1.0000	0.1005	-0.0968	0.1060
il1ra	-0.0161	0.1606	0.0357	-0.0339	0.1005	1.0000	0.1346	-0.1493
mcp1	0.0306	0.0382	0.0540	0.0717	-0.0968	0.1346	1.0000	-0.0504
mip1a	0.1596	-0.0116	-0.0735	0.1435	0.1060	-0.1493	-0.0504	1.0000
mip1b	-0.0718	0.0284	0.0466	0.1706	-0.0083	0.1932	-0.0047	-0.0429
mmp9	0.0565	-0.3784	0.0530	-0.1015	0.0154	0.1170	-0.1562	0.1252
sap	0.1019	0.1178	-0.0665	-0.0131	-0.0153	-0.0719	-0.0952	-0.1388
tnfa	-0.0877	-0.1331	-0.0523	0.1446	0.0749	-0.0383	0.0466	0.0646
vegf	0.0257	0.1473	0.0035	0.0308	-0.0525	-0.0303	-0.0545	0.1597
scd401	0.1618	-0.0090	0.0296	-0.0950	0.0383	0.0550	-0.0610	-0.0665
	mip1b	mmp9	sap	tnfa	vegf	scd401		
ddimer	-0.0718	0.0565	0.1019	-0.0877	0.0257	0.1618		
fractalkine	0.0284	-0.3784	0.1178	-0.1331	0.1473	-0.0090		
ifng	0.0466	0.0530	-0.0665	-0.0523	0.0035	0.0296		
il10	0.1706	-0.1015	-0.0131	0.1446	0.0308	-0.0950		
il1a	-0.0083	0.0154	-0.0153	0.0749	-0.0525	0.0383		
il1ra	0.1932	0.1170	-0.0719	-0.0383	-0.0303	0.0550		
mcp1	-0.0047	-0.1562	-0.0952	0.0466	-0.0545	-0.0610		
mip1a	-0.0429	0.1252	-0.1388	0.0646	0.1597	-0.0665		
mip1b	1.0000	-0.0202	-0.2410	0.0738	-0.0222	-0.1490		
mmp9	-0.0202	1.0000	-0.0813	-0.1197	-0.0141	-0.0332		
sap	-0.2410	-0.0813	1.0000	0.0074	0.0263	-0.0976		
tnfa	0.0738	-0.1197	0.0074	1.0000	-0.0710	-0.0798		
vegf	-0.0222	-0.0141	0.0263	-0.0710	1.0000	0.0159		

```
scd401      -0.1490 -0.0332 -0.0976 -0.0798  0.0159  1.0000
```

No, there are very weak covariances between variables. PCA aims to maximise variance. It does this by loading strongly correlated variables onto the same components. When variables are not strongly correlated, PCA performs poorly.

Part d

```
# Extract response variables:
pcr_dat = data_tidy_lum_response[,-(1:2)]

# Calculate correlation and covariance matrix:
cov_mat = cov(pcr_dat)
cor_mat = cor(pcr_dat)

# Eigen decomposition:
eig_mat = eigen(cov_mat)
eig_val = eig_mat$values
eig_vec = as.matrix(eig_mat$vectors)

# The eigen vectors are the loadings
# We want 2 principal components -> take the first 2 columns

# Calculate loadings:
loadings = eig_vec[,1:2]

# Calculate scores:
X          = as.matrix(pcr_dat)
scores     = X%*%loadings

# Regress:
Y = as.matrix(data_tidy_lum_response$crp)
X = cbind(1,scores)

beta_hat = solve(t(X)%*%X)%*%t(X)%*%Y
rownames(beta_hat) = c('intercept','PC1','PC2')
colnames(beta_hat) = c('crp')
round(beta_hat,4)
```

```
              crp
intercept 0.5184
```

```
PC1      -0.1172
PC2       0.0147
```

```
# Calculate the proportion of variation explained:
tot_var  = sum(eig_val)
prop_var = eig_val/tot_var
cum_var  = cumsum(prop_var)
round(cum_var[2],4)
```

```
[1] 0.3603
```

Question 3.3

In `data_tidy_lum_response`, the variables `crp` to `il1ra` are one set of variables ($\mathbf{X}^{(1)}$), and the variables `mcp1` to `scd401` are another set ($\mathbf{X}^{(2)}$).

Standardise the variables in `data_tidy_lum_response`, creating a new matrix `lum_response_mat_std`, using this code, and use `lum_response_mat_std` for question 3.3:

```
if (!exists("data_tidy_lum")) {
  stop("Please run code above to create the `data_tidy_lum` object.")
}
lum_response_mat_std <- data_tidy_lum |>
  dplyr::select(crp:scd401) |>
  dplyr::mutate(across(dplyr::everything(), ~ (. - mean(.))/sd(.))) |>
  as.matrix()
```

Part a

Okayyyyyyy, CCA from first principles... here we go

```
X1 = lum_response_mat_std[,1:7]
X2 = lum_response_mat_std[,8:15]

# STEP 1: Obtain correlations matrix:
S   = cov(lum_response_mat_std)

# STEP 2: Partition correlation matrix:
S11 = S[1:7,1:7]
S12 = S[1:7,8:15]
S21 = S[8:15,1:7]
```

```

S22 = S[8:15,8:15]

# STEP 3: Calculate intermediate matrices:
S11_inv = solve(S11)
S22_inv = solve(S22)

S11_eig = eigen(S11)
S22_eig = eigen(S22)

S11_inv_sqrt = S11_eig$vectors%%diag(1/sqrt(S11_eig$values))%%t(S11_eig$vectors)
S22_inv_sqrt = S22_eig$vectors%%diag(1/sqrt(S22_eig$values))%%t(S22_eig$vectors)

# STEP 4: Calculate R and eigen decomposition:
R = S11_inv_sqrt%%S12%%S22_inv%%S21%%S11_inv_sqrt
R_eig = eigen(R)

# STEP 5: Obtain canonical correlations:
can_cor = sqrt(R_eig$values)
round(can_cor[1],4)

```

```
[1] 0.5186
```

```

# STEP 6: Obtain canonical coefficients:
A = S11_inv_sqrt%%R_eig$vectors
round(A[,1],4)

```

```
[1] -0.0208 -0.0442  0.9264  0.0466  0.2474  0.1230 -0.5372
```

```

sf_A = sqrt(diag(t(A)%%S11%%A))
A      = A%%diag(1/sf_A)
round(A[,1],4)

```

```
[1] -0.0208 -0.0442  0.9264  0.0466  0.2474  0.1230 -0.5372
```

```

# STEP 7: Obtain B (proportional):
B_p = S22_inv%%S21%%A
round(B_p[,1],4)

```



```

      mcp1    mip1a    mip1b    mmp9      sap    tnfa    vegf    scd401
-0.0798  0.1560 -0.0108 -0.4785  0.0986 -0.1146  0.1054 -0.0764

```

```

# STEP 8: Standardise B:
sf_B = sqrt(diag(t(B_p)%*%S22)%*%B_p))
B     = B_p%*%diag(1/sf_B)
round(B[,1],4)

```

```

      mcp1    mip1a    mip1b    mmp9      sap    tnfa    vegf    scd401
-0.1539  0.3009 -0.0208 -0.9228  0.1902 -0.2211  0.2032 -0.1473

```

This was just extra:

```

# STEP 9: Canonical loadings:
loadings_U = t(A)%*%S11
loadings_U

```

```

      crp      ddimer fractalkine      ifng      il10      il1a
[1,]  0.03252222  0.02467724  0.81436513 -0.16387741  0.20834738 -0.005800059
[2,]  0.79175275  0.57385951 -0.08212957 -0.07832526  0.08243119  0.177634458
[3,]  0.05506480  0.09579866  0.40153459  0.27780512  0.27161560 -0.323916518
[4,]  0.10718078  0.25105061  0.28562584  0.15917734 -0.85925619 -0.293106274
[5,]  0.53523744 -0.74362941 -0.05515132  0.16280910 -0.09845696 -0.128716557
[6,] -0.08331488 -0.04773175  0.18577734  0.15949194 -0.35651628  0.865405310
[7,]  0.25323812 -0.20642066  0.22292852 -0.90141653 -0.03004326  0.109931147
      il1ra
[1,] -0.38005369
[2,] -0.14769986
[3,]  0.75748162
[4,]  0.01872842
[5,] -0.11528144
[6,]  0.37536228
[7,]  0.32469699

```

```

loadings_V = t(B)%*%S22
loadings_V

```

```

      mcp1    mip1a    mip1b    mmp9      sap    tnfa
[1,] -0.05540231  0.1955900 -0.05910519 -0.84759119  0.26123262 -0.10117138

```

```

[2,] -0.10578995  0.8266510 -0.08156432  0.36431796  0.04123841 -0.31605093
[3,]  0.53105322 -0.1776096  0.66832008 -0.07395748 -0.10668996 -0.31433039
[4,] -0.04509193 -0.4229722 -0.53761713  0.07291309  0.21539530 -0.73660660
[5,] -0.27710727  0.1355048  0.20051456  0.19725288 -0.55830915 -0.40228253
[6,] -0.51548086 -0.1289986  0.35288702 -0.17420677 -0.34482528 -0.08505775
[7,] -0.46523195 -0.1792388  0.29345232  0.20684183  0.65751150 -0.15890043
      vegf      scd40l
[1,]  0.2915168 -0.12184761
[2,]  0.1366069  0.24466241
[3,]  0.2088991  0.07334941
[4,]  0.1206388  0.35253514
[5,]  0.2193337 -0.51225960
[6,] -0.3011638  0.58352804
[7,] -0.1817061 -0.38020360

```

STEP 10: Cross-loadings:

```

cross_U = t(A)%*%S12
cross_U

```

```

      mcp1      mip1a      mip1b      mmp9      sap      tnfa
[1,] -0.02872888  0.101423245 -0.03064901 -0.439518620  0.13546224 -0.052462443
[2,] -0.04706740  0.367788382 -0.03628909  0.162090067  0.01834754 -0.140615402
[3,]  0.17645804 -0.059016011  0.22206899 -0.024574545 -0.03545087 -0.104445512
[4,] -0.01318409 -0.123669678 -0.15718984  0.021318511  0.06297781 -0.215370871
[5,] -0.05267817  0.025759499  0.03811787  0.037497827 -0.10613472 -0.076474019
[6,] -0.04662083 -0.011666818  0.03191561 -0.015755510 -0.03118649 -0.007692745
[7,] -0.01630288 -0.006280973  0.01028330  0.007248251  0.02304083 -0.005568265
      vegf      scd40l
[1,]  0.151166116 -0.06318411
[2,]  0.060778291  0.10885367
[3,]  0.069412872  0.02437250
[4,]  0.035272661  0.10307510
[5,]  0.041695390 -0.09738069
[6,] -0.027237690  0.05277511
[7,] -0.006367432 -0.01332328

```

```

cross_V = t(B)%*%S21
cross_V

```

```

      crp      ddimer  fractalkine      ifng      il10

```

```

[1,] 0.016864407 0.012796388 0.422289239 -0.08497867 0.10803858
[2,] 0.352261681 0.255317980 -0.036540573 -0.03484798 0.03667477
[3,] 0.018296899 0.031831921 0.133421670 0.09230892 0.09025227
[4,] 0.031337782 0.073402801 0.083511994 0.04654067 -0.25123146
[5,] 0.101748783 -0.141364153 -0.010484280 0.03095005 -0.01871669
[6,] -0.007535117 -0.004316928 0.016801969 0.01442468 -0.03224385
[7,] 0.008874092 -0.007233492 0.007811968 -0.03158787 -0.00105279
      illa      illra
[1,] -0.003007622 -0.197076935
[2,] 0.079032011 -0.065713696
[3,] -0.107630783 0.251695530
[4,] -0.085699142 0.005475861
[5,] -0.024469053 -0.021915033
[6,] 0.078268500 0.033948304
[7,] 0.003852260 0.011378188

```

```
library(CCA)
```

Loading required package: fda

Loading required package: splines

Loading required package: fds

Loading required package: rainbow

Loading required package: MASS

Attaching package: 'MASS'

The following object is masked from 'package:dplyr':

```
select
```

Loading required package: pcaPP

Loading required package: RCurl

Loading required package: deSolve

Attaching package: 'fda'

The following object is masked from 'package:graphics':

matplot

The following object is masked from 'package:datasets':

gait

Loading required package: fields

Loading required package: spam

Spam version 2.11-4 (2026-05-28) is loaded.

Type 'help(Spam)' or 'demo(spam)' for a short introduction and overview of this package.

Help for individual functions is also obtained by adding the suffix '.spam' to the function name, e.g. 'help(chol.spam)'.

Attaching package: 'spam'

The following objects are masked from 'package:base':

backsolve, forwardsolve

Loading required package: viridisLite

Loading required package: RColorBrewer

Try help(fields) to get started.

```
# Check using built in functions:
```

```
cca_fit1 = cc(X1,X2)
```

```
# Canonical correlation:
```

```
round(cca_fit1$cor[1],4)
```

```
[1] 0.5186
```

```
# Canonical coefficients
round(cca_fit1$xcoef[,1],4)
```

```
      crp      ddimer fractalkine      ifng      il10      il1a
-0.0208   -0.0442      0.9264      0.0466      0.2474      0.1230
      il1ra
-0.5372
```

```
round(cca_fit1$ycoef[,1],4)
```

```
      mcp1      mip1a      mip1b      mmp9      sap      tnfa      vegf      scd401
-0.1539   0.3009  -0.0208  -0.9228   0.1902  -0.2211   0.2032  -0.1473
```

Part b

Part c

Question 3.4

Part a

Okay, LDA from first principles. Here we go :)

```
# First, let's actually get the data:
dat_og = data_tidy_lum[,-(1:6)]
dat = data_tidy_lum[,-(1:6)]
dat$age_infxn = as.numeric(as.factor(dat$age_infxn))
X = as.matrix(dat[, -1])

# STEP 1: Get everything we actually need to do this
g = length(unique(dat$age_infxn)) # no. groups
n_i = table(dat$age_infxn)        # no. obs per group
bar_overall = colMeans(X)         # column means
bar_group = dat |>               # group mean
  dplyr::group_by(age_infxn) |>
  dplyr::summarise(
    dplyr::across(everything(), mean),
    .groups='drop'
  ) |>
```

```

dplyr::select(-age_infxn) |>
as.matrix()

# STEP 2: Calculate B
B = Reduce(`+`,lapply(1:g,function(i){
  n_i[i]*(bar_group[i,]-bar_overall)%*%t(bar_group[i,]-bar_overall)
}))

# STEP 3: Calculate W
W = Reduce(`+`,lapply(1:g,function(i){
  Xi = X[dat$age_infxn==i,]
  cov(Xi)*(n_i[i]-1)
}))

# STEP 4: Eigen decomposition
mat = solve(W)%*%B
eig_mat = eigen(solve(W)%*%B)
eig_vec = eig_mat$vectors |> Re()

S_pooled = W/(sum(n_i)-g)

sd_vec = diag(t(eig_vec)%*%S_pooled%*%eig_vec) |> sqrt()

sol = eig_vec%*%diag(1/sd_vec)
sol = sol[,1:2]

# STEP 5: Display results
rownames(sol) = colnames(X)
colnames(sol) = c('LD1','LD2')
knitr::kable(
  round(sol,4))

```

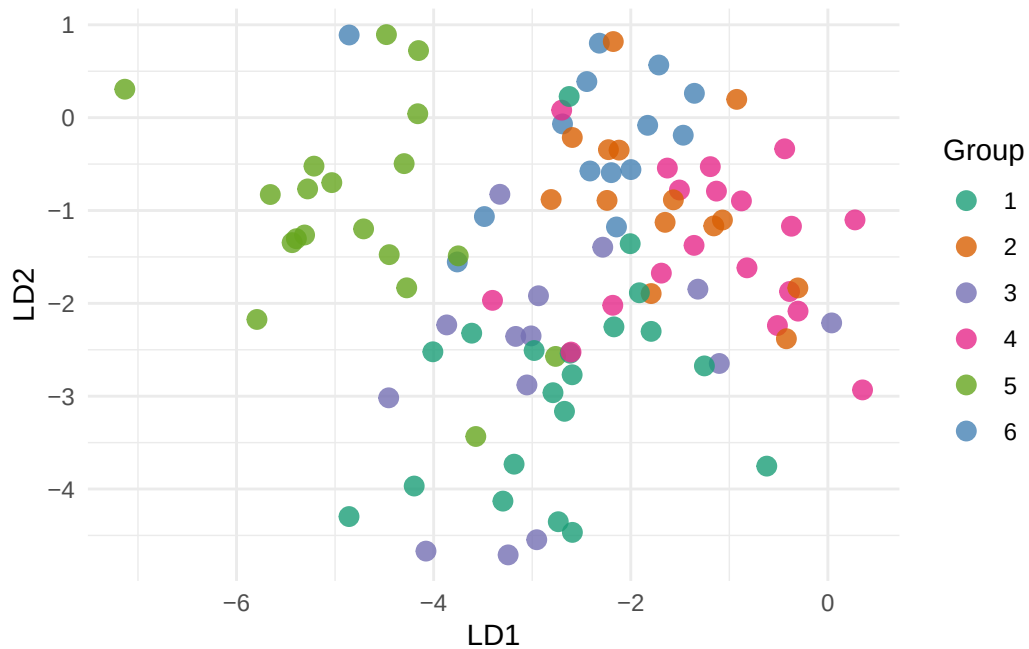
	LD1	LD2
crp	-0.0060	-0.0194
ddimer	-0.9689	1.0818
fractalkine	-0.7345	-0.5901
ifng	-0.2762	0.3395
il10	-0.1122	0.1026
il1a	-0.2971	-0.1076
il1ra	0.2205	-0.1495
mcpl	-0.4602	-0.0320

	LD1	LD2
mip1a	0.1291	0.1120
mip1b	0.0901	0.0709
mmp9	0.7858	0.5831
sap	0.0966	-0.0375
tnfa	-0.2289	0.1267
vegf	-0.6852	-0.0429
scd40l	0.1747	0.1959

```
# STEP 6: Plot
dat$LD1 = as.numeric(as.matrix(X)%*%(sol[,1]))
dat$LD2 = as.numeric(as.matrix(X)%*%(sol[,2]))

col_vec <- c(
  "#1b9e77",
  "#d95f02",
  "#7570b3",
  "#e7298a",
  "#66a61e",
  'steelblue'
) |>
  toupper()

ggplot(data=dat,
       mapping=aes(x=LD1,y=LD2,colour=as.factor(age_infxn)))+
  geom_point(size=3,
             alpha=0.8)+
  theme_minimal()+
  scale_colour_manual(
    values=col_vec,
    name="Group"
  )
```



```
library(MASS)

# Fit model using MASS:lda() function:
lda_fit = lda(age_infxn~.,data=dat_og)

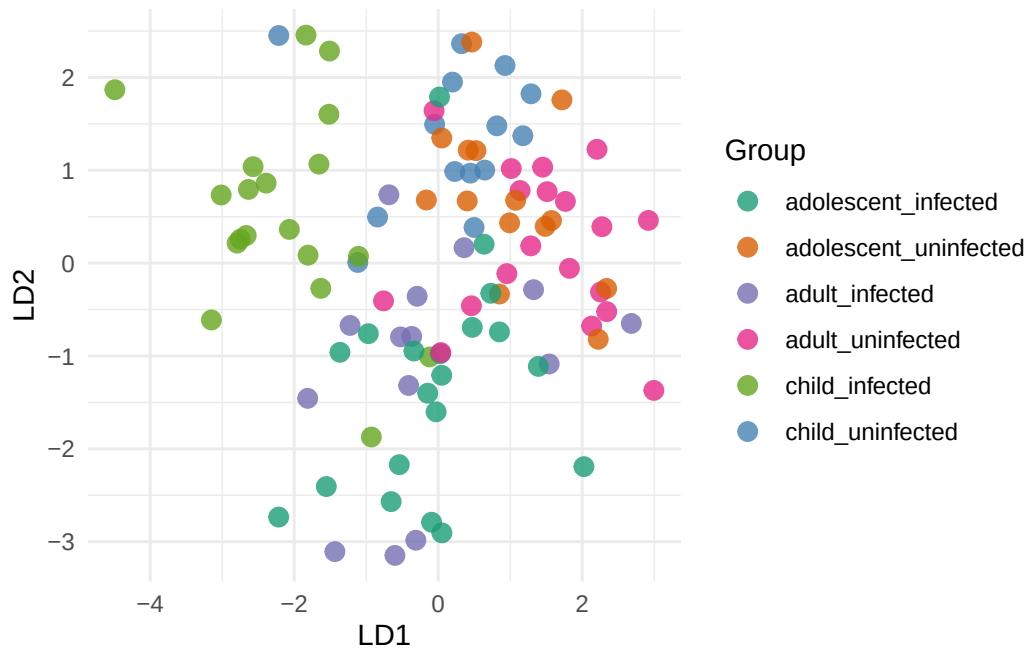
# Make predictions:
lda_pred = predict(lda_fit)

# Calculate scores:
scores_df = data.frame(
  group=dat_og$age_infxn,
  lda_pred$x[,1:2]
)

# Plot:
ggplot(data=scores_df,
       mapping=aes(x=LD1,y=LD2,colour=factor(group)))+
  geom_point(size=3,
             alpha=0.8)+
  theme_minimal()+
  scale_colour_manual(
    values=col_vec,
```



```
name="Group"
)
```



Part b

All that has to be mentioned here is which groups LD1 and LD2 split on as well as which variables load strongly onto those linear discriminants.

Multiple choice

- 4.1:
- 4.2:
- 4.3:
- 4.4:
- 4.5:
- 4.6:
- 4.7:
- 4.8: